

Whitebox2.0-用户使用说明书

 Whitebox是一个基于testng进行二次开发，完全**数据驱动**的本地测试框架，能支持用户连接真实的代码依赖组件，不需要部署服务即可进行本地代码测试。

 欢迎提需求，直接找他：  **李昌安**

可以在用户群反馈



AICO-TEST - AI Testing工具包用户群

AICO-TEST工具用户群

群名片

1. 定位

从自动化测试金字塔的视角看whitebox： [服务端自动化测试金字塔-一个案例说明](#)

 同步块 [该内容不支持导出查看]

- 单元测试（UT）
- 单接口自动化
 - whitebox测试（依赖mock，本地环境），注重的是服务自身逻辑的测试
 - ATE单接口测试（不强依赖Mock，真实boe环境）
- 场景测试（多接口多服务少量Mock，BOE/PPE/PROD）
- E2E测试，端到端测试，从用户操作界面发起的场景测试（Pagepass*AI）

whitebox相对来说**更关注服务内部逻辑的质量保障**，如果你的项目有以下痛点，那么你**合适用** whitebox：

1. 代码层级多，PSM关系复杂，开发自测、提测的相互block
2. 代码业务复杂，依赖众多，本地启动服务用postman测试成本较高，而且测试不能留痕
3. 代码依赖接口较多，且一旦有需求大多依赖接口也得改，自测需要通过mock的方式做自测
4. MQ和cronjob的服务，whitebox提供了较好的能力支持

如果你的项目更多是上下游调用较多的转发层服务，包括以下类型的，那么不适用whitebox（推荐用ATE）：

- 1. 业务相对简单，依赖较少，能够灵活部署自测的服务
- 2. 代码依赖接口较多，但本身业务主要是调用方，且依赖接口修改不频繁，对于调用方来说是"基础设施"，可以直接在BOE环境调用，可以很方便在BOE环境自测
- 3. 主要以HTTP接口为主，对接页面内容较多，主要是作为转发层的服务

whitebox是一个不依赖部署，可直接在本地进行接口级别测试的质量保障工具，引入whitebox主要是更好解决质量问题，不依赖部署就可以有很好测试交付灵活性，而是否能增加开发效率这个不敢保证。

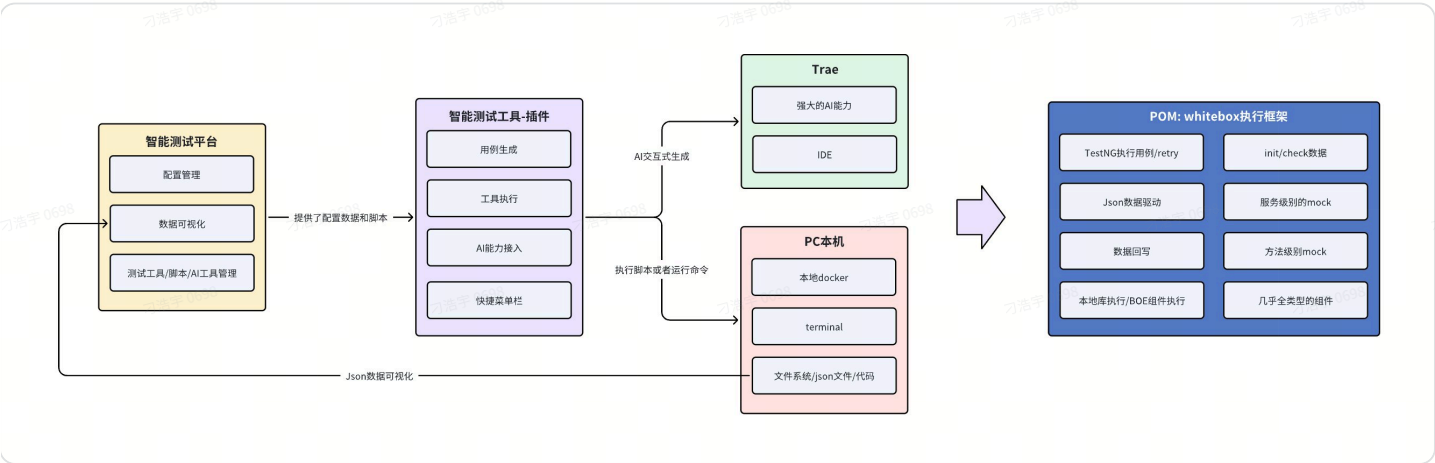
2. 框架说明

- 框架配置说明：[Whitebox-集测框架配置说明](#)
- JSON格式说明：[Whitebox JSON功能完整说明文档](#)

同步块 [该内容不支持导出查看]

whitebox框架主要分三部分内容

- 智能测试平台页面：主要做一些测试配置数据的管理、测试工具的管理（链接地址：[智能测试平台](#)）
- 智能测试工具插件（whitebox-generator）：主要是沟通了平台数据、本地代码、本地环境、Trae的AI能力的工具，能进行工具执行、一键安装、prompt管理等工作
- whitebox执行框架：是一个pom包，引入之后能够执行whitebox的json格式的用例case，并且能够连接各类数据源和通过testng的retry能力进行快速执行



whitebox执行框架，整体架构如下

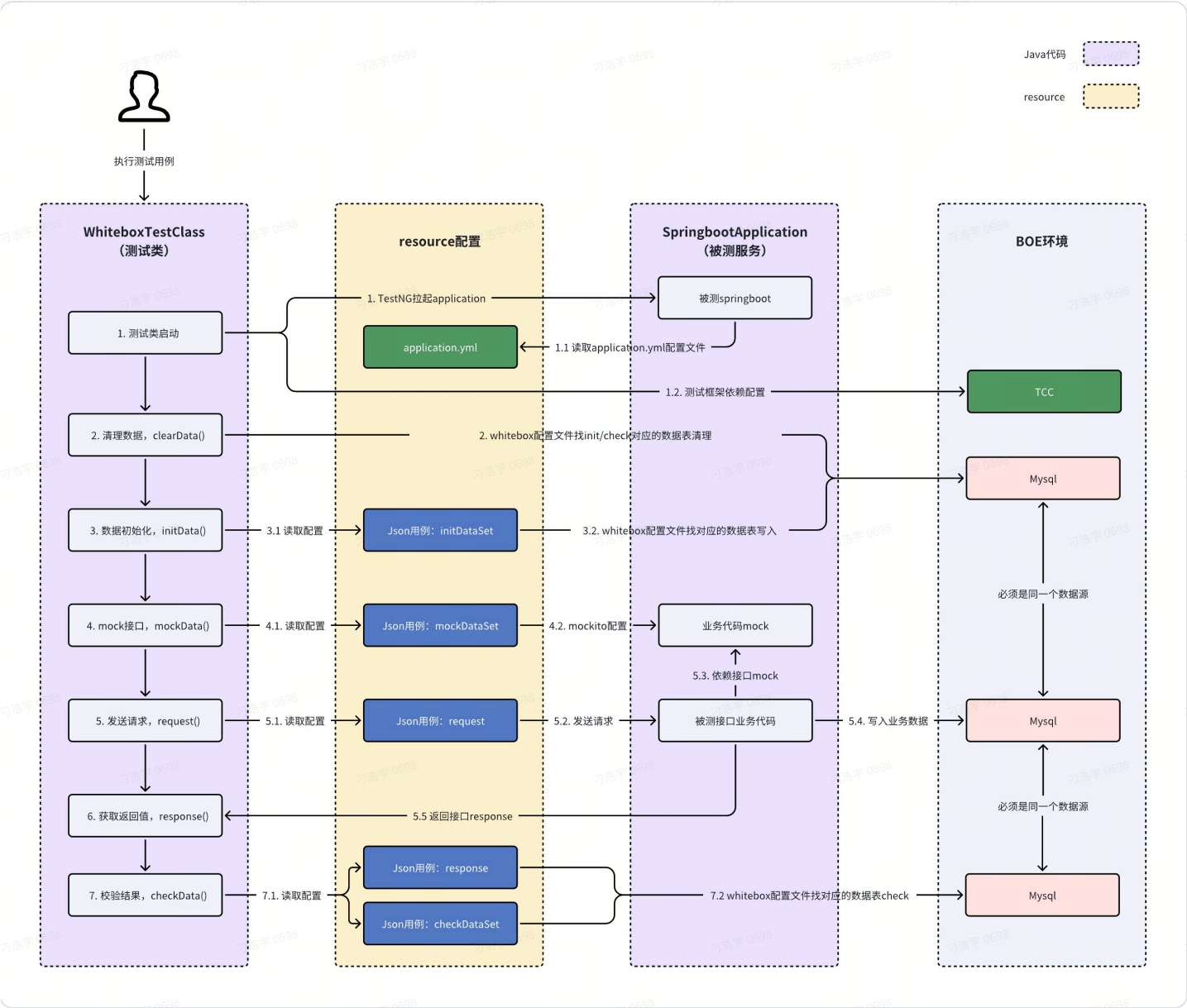
3. whitebox功能介绍

3.1 基础能力



参考上面的类关系，我们先用一个简单的用例（是只有数据库类型数据初始化和写入的thrift接口），整体介绍一下框架的基础能力。

更完整能力可以查看：[Whitebox-集测用例Json能力概览](#)



- 被测服务启动**，由于现在whitebox是基于testng做二次开发的，所以服务拉起完全遵守的testng流程，启动测试类依赖springboot启动配置文件和whitebox框架配置文件
- 数据清理**，whitebox执行第一步是进行数据清理，会把init的数据和check的数据执行一次清理动作

- **数据初始化**，读取的json文件的initDataSet的数据信息，把数据写入到指定的数据库内
- **mock接口信息**，读取json文件的mockDataSet的数据信息，通过mockito进行mock，在后面代码执行的时候会拦截匹配到的mock请求
- **发送请求**，实际上因为已经testng拉起了被测服务，这里的发起请求是调用的服务的入口方法（如果是http、cronjob、mq就是调用入口方法）；在用例的请求发送的过程中
 - 读取上述初始化的数据
 - 调用上述被mock的方法
 - 写入业务流程中产生的数据
 - 构造返回值
- **获取返回值**，得到返回值信息
- **结果校验**，会对获取到的response信息和读取json文件中的checkDataSet进行数据校验，也可以自定义在测试类里面进行数据校验

上述只是一个thrift接口的用例说明，whitebox的框架能力都主要是在json配置信息中体现的可以参考下面这个文档

Json格式详细说明：[📖 Whitebox-集测用例Json能力概览](#) 【推荐指数：★★★★★】

- 以下是whitebox支持的测试类型

测试类	支持情况
thrift	框架默认支持，测试类可配置，json配置语法有相应适配，mock配置原生支持rpc类型
mq	框架默认支持，测试类可配置，json配置语法有相应适配
cronjob	框架默认支持，测试类可配置，json配置语法有相应适配
http	支持直接在测试类里调用入口方法的方式测试，json适配较少，json的mock配置暂不支持支持http类型，只能通过方法级mock对http的入口方法进行mock

- 以下是whitebox原生支持的连接的BOE组件

组件类型	支持情况
RDS	框架默认支持，json可配置，key是 <i>initDataSet</i> 、 <i>checkDataSet</i>
redis	框架默认支持，json可配置，key是 <i>initRedisDataSet</i> 、 <i>checkRedisDataSet</i>

rmq	框架默认支持，json可配置，key是mqDataSet
bmq	框架默认支持，json可配置，key是byteMqDataSet
bytekv	框架默认支持，json可配置，key是initByteKVSet、checkByteKVSet
其他组件	可以通过测试类编写java代码的方式支持

3.2 进阶能力（whitebox2.0提效专项）

 whitebox2.0的建设内容，主要围绕的用例编写提效、快速接入的能力进行，建设工作详见：
[📖 Whitebox One Page\[持续更新\]](#)

- 状态：待建设，验证中，已交付

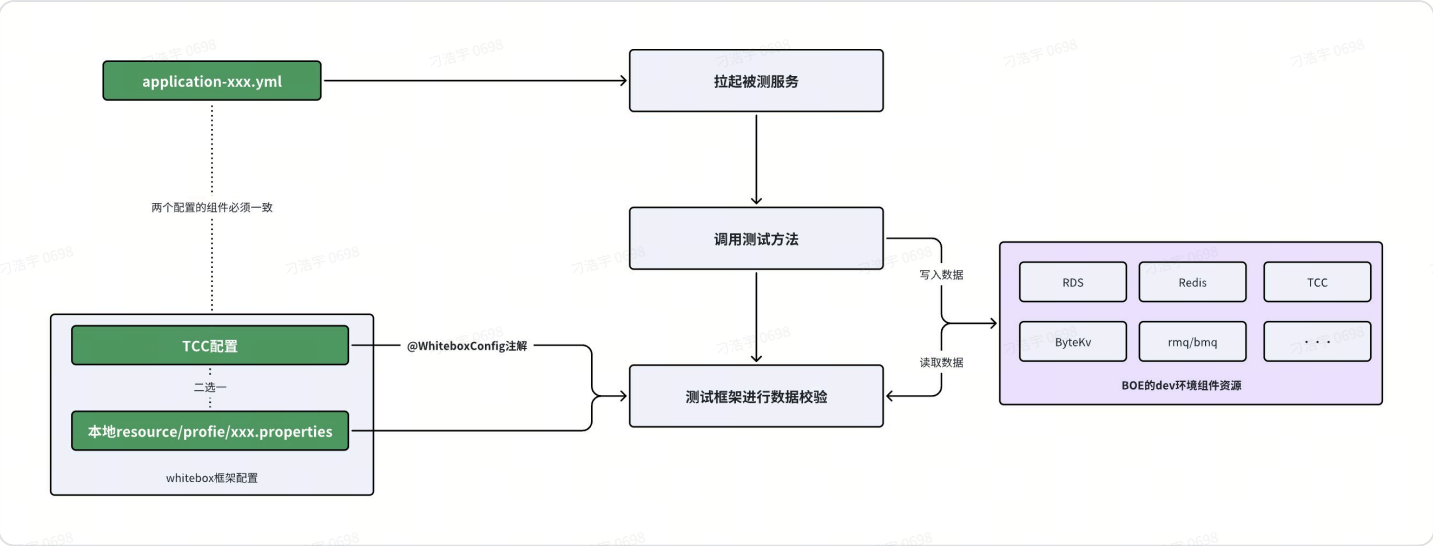
分类	能力	功能描述	预期提效级别	使用方式	注意事项
用例编写优化	平台生成用例框架	平台生成的用例框架	★★★★	1. 平台生成用例框架 a. 平台功能，从用例配置开始 b. 流程说明：第一个用例如何构造	
	数据回写能力	数据回写check、mock等数据信息	★★★★★★	2. 回写数据：📖 Whitebox2.0-writeback功能说明 a. 数据库数据回写 b. mq、redis数据回写 c. mock数据回写	使用回写能力一定要注意，回写的数据不是100%准确的，注意不要happytest!
	复杂场景的用例构建	- 当场景用例转存数据子模板 - 多个子模板追加完成复杂用例	★★★★ 目前这个子模板的还不能完美追加，待优化	1. 追加用例子模板构建复杂用例	目前可用性稍差

执行效率优化	springboot 热加载能力	解决重启 spring 问题； 加快执行速度	★★★★★ 大幅提升执行速度	1. ☒ 不重启 springboot 进行执行： whitebox-秒级执行配置方法（尝鲜版） ；	
	本地数据库 一键部署	主要是支持 whitebox 能够进行本地数据库构建的能力	★★★★★ 大幅提升执行速度	1. ☒ 使用本地库进行执行： 【使用说明】 本地组件跑 whitebox 一键部署脚本-精简版 2. ☒ 仓库需要有本地数据库所需的 sql 文件： 本地数据库 sql 文件创建	
生态场景建设和体验优化	whitebox 的 trae 插件能力	联通多部分能力； 具备多种工具能力； 联通了 Trae AI 的能力	★★★★★ 由于 whitebox 大多数功能需要增加一些配置、命令、脚本完成，有一定使用成本，因此提供了插件能力支持一键完成	1. ☒ 更友好的交互方式，和减少上手成本【 智能测试工具 】 Whitebox-Generator 插件 a. 一键接入能力 b. 一键部署的脚本 c. 一键切换 env 配置能力 d. 一键安装 MCP 能力 e.	
	UI 可视化的更方便的 case review 能力	实现多种用例查看、校验功能	★★★★ 解决用例 review 时间	1. ☒ 已交付页面读取本地代码的能力，支持在页面更好的 review case 【Whitebox 插件】在测试平台上可视化查看测试 case 2. ☐ 进一步增加页面可视化的效果的能力，以及交互能力提升	
	AI 能力建设	通过 AI 的能力进一步的减少重复的工作	★★★★ 基于插件能够很好连接到本地代码和 AI 的特点，能够做更多 AI 的周边能力	1. ☒ 集成 BAM、RDS 的 MCP 能力，通过 MCP 生成用例 2. ☐ AI review 的能力建设 3. ☐ AI 生成 init 和 request 信息 4. ☐ 汇总本地代码生成业务知识库信息	

4. whitebox的执行说明

4.1 执行说明

上述其实已经提到了一部分whitebox的执行用例相关的说明和一些工作，但是一个用例如何执行的，是需要了解的，不然往往会在运行whitebox用例出错之后无从排查。



whitebox的运行主要分为3个阶段

1. **拉起被测服务**，这部分主要关注在被测服务的启动配置文件是否是正确的配置信息
2. **执行测试代码**，这部分需要保证，执行测试代码的依赖方法能够被调通或者已经配置了mock(大概率是只能被mock的)
3. **框架层进行数据校验**，框架层在执行了业务代码之后，需要连接业务代码的同一个组件，比如mysql、redis等组件，获取信息进行数据校验，这就需要要求框架的配置文件需要和被测服务的配置文件，配置组件必须是同一个；框架层的配置文件当前分两种：
 - a. 使用@WhiteboxConfig注解，表示使用的TCC配置文件，参考：[whitebox-通过tcc读取集测配置](#)
 - b. 不使用@WhiteboxConfig注解，就会正常找本地的resource配置文件

对应的，我们跑用例就有以下几种玩法

运行方式	说明	启动类配置	whitebox配置	环境变量
------	----	-------	------------	------

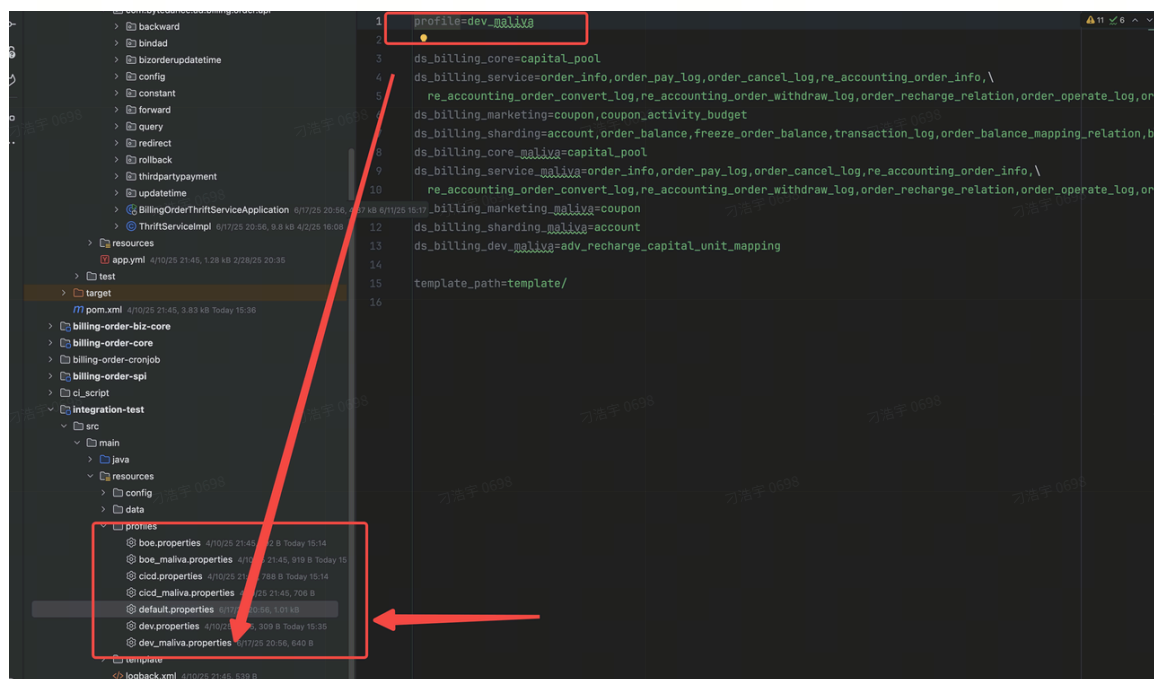
dev运行	连接的BOE组件，需要搭建有一套可以在BOE正常运行的依赖环境，包括BOE数据库、redis等	配置链接BOE环境的配置	- 使用@WhiteboxConfig配置的话，配置是dev或者devi18n - 如果使用本地配置文件，则需要保证本地配置文件和启动类配置一致	参考： 环境变量一览 按需配置环境变量
cicd运行	与dev不同的是，cicd运行的数据库都是跑的127.0.0.1:3306的，因此需要在同机器（容器或者PC机）上启动一个mysql服务	- 数据库需要配置127.0.0.1的配置 - 其他组件配置BOE环境的配置	- 使用@WhiteboxConfig配置的话，配置是cicd或者cicdi18n - 如果使用本地配置文件，则需要保证本地配置文件和启动类配置一致	参考： 环境变量一览 按需配置环境变量
cicd运行-国内服务发现	主要是非中业务的玩法，由于使用海外的服务发现不稳定，可以用海外的服务发现跑用例，但是一般这种服务的依赖只有数据库（本地）、redis、mq这种简单的组件，即除了数据库外的组件，在中国区BOE得有相应同名PSM配套的组件	同cicd	同cicd	参考： 环境变量一览 按需配置环境变量，这里使用的是国内的服务发现配置，跑非中业务
local运行方式	目前local的运行方式，其实就是cicd的运行方式	同cicd	同cicd	参考： 环境变量一览 按需配置环境变量

4.1 配置文件分类

详细的参考配置文件说明：[Whitebox-测试类配置文件说明](#)

whitebox的执行过程，是强依赖几个配置文件的，而whitebox出错往往都是这几个配置文件没有正确配置导致的。测试框架的配置文件主要有cicd和dev，包括一些历史的玩法，这个配置文件可能有两种存在方式

- **新的配置方法：**TCC配置的方式，需要依赖@WhiteboxConfig注解来定义到底读取的哪个配置文件信息；通过本地的环境变量信息来确定访问的哪个region的TCC，比如配置的中国区的开发机做服务发现，那么使用的就是中国区的TCC
- **原先的配置方法：**直接把whitebox框架的配置文件写在代码仓库的profile文件夹里，类似下图；这种配置文件方式不方便维护，但是比较方便调试，后续会修改为local的能力作为支持



- [后续建设]除了cicd和dev的配置文件，还有local的配置文件，即原先的本地化的配置文件，主要是用来支持初建建立用例的时候，TCC配置频繁 变更的调试状态

4.2 环境变量问题

whitebox环境变量问题说明：[环境变量问题说明&怎么配置变量](#)

内外同学的网络环境区别：[国外base同学试用whitebox注意事项](#)

5. whitebox接入指南



AI接入：[Whitebox-用户通过skills接入说明（whitebox-setup）](#)

6. whitebox的skills能力

whitebox的skills是整合了whitebox现有的完整能力的skills，包含 Whitebox 数据驱动集成测试框架的完整 Skill 体系，覆盖从项目接入、知识库构建、用例生成、格式校验、运行到自愈的**全生命周期**。

whitebox的skills总体说明参考：[📖 whitebox-generatorWhitebox测试执行引擎Skills专辑说明](#)

7. 用户常见问题和踩坑文档

踩坑文档：[📖 Whitebox用户框架踩坑记录](#) 【推荐指数★★★★★】

项目实践：[📖 项目中试用whitebox新功能记录](#) 【推荐指数★★★★★】

环境变量配置：[📖 EnvFile插件简化环境变量](#) 【推荐指数★★★★★】

spring2.0使用whitebox：[📖 SpringBoot2.0使用集成测试框架](#) 【推荐指数★★★★★】